CalderR.

RETHINKING · HOW · WORK · WORKS

Agentic AI Engineering Internship — 2026

WEEK 3

Embeddings, RAG & Vector Databases

Monday 6 July – Friday 10 July 2026

Building systems that know things — retrieval-augmented generation at scale

Week Overview

Week 3 introduces the technology that lets AI systems work with knowledge they were never trained on. Interns learn how embeddings represent meaning, how vector databases store and retrieve it efficiently, and how to build complete retrieval-augmented generation (RAG) pipelines that are measurably evaluated rather than assumed to work.

Detail	Information
Week Dates	Monday 6 July – Friday 10 July 2026
Theme	Embeddings, RAG & Vector Databases
Primary Stack	Python · LangChain · ChromaDB · FAISS · sentence-transformers · RAGAS
Total Commitment	20 hours (4 hours/day · 5 days)
Deliverable Due Friday	1 Intermediate Project + 1 Production Project (choose one of each)

Learning Objectives

- Understand embeddings: what they represent and how they are computed
- Build complete RAG pipelines from ingestion to generation
- Work with ChromaDB, FAISS, and Qdrant vector databases
- Implement advanced retrieval: hybrid search, re-ranking, metadata filtering
- Evaluate RAG systems using the RAGAS framework
- Build a production RAG application with multiple document sources

Topics & Subtopics

Topic	Subtopics
Embeddings	Semantic similarity, cosine distance, embedding models (sentence-transformers, text-embedding-3), dimensionality, batching
Document Processing	Text splitting strategies, chunk size tuning, overlap, metadata extraction, document loaders
Vector Databases	ChromaDB (local), FAISS (in-process), Qdrant (server), index types (HNSW), persistence, namespaces
RAG Architecture	Naive RAG, advanced RAG, modular RAG, ingestion pipeline vs query pipeline
Advanced Retrieval	Hybrid search (BM25 + semantic), re-ranking (cross-encoder), HyDE, multi-query, parent-document retrieval

Topic	Subtopics
RAG Evaluation	RAGAS metrics: faithfulness, answer relevancy, context precision, context recall. TruLens basics.

Daily Breakdown

Each day is structured around a core learning block followed by applied, hands-on practice. Complete each day's practice before moving to the next — concepts compound throughout the week.

Day	Focus	Core Learning	Applied Practice
Mon	**Embeddings Deep Dive**	Study embedding theory: vector spaces, cosine similarity, semantic meaning. Install sentence-transformers. Visualize embeddings with 2D PCA.	Lab: embed 100 Wikipedia sentences. Build a semantic search CLI. Compare results of different embedding models (all-MiniLM-L6-v2 vs bge-small-en).
Tue	**Vector DBs**	ChromaDB tutorial: create collection, add docs, query. FAISS intro: IndexFlatL2 vs IndexIVFFlat. Study tradeoffs: in-memory vs persistent vs server.	Build: Document ingestion pipeline for 50 PDF pages. Store in ChromaDB with metadata (source, page, date). Query with filters.
Wed	**RAG Pipeline**	Study RAG paper. Build naive RAG with LangChain: load → split → embed → store → retrieve → generate. Tune chunk size (256/512/1024 tokens).	Evaluate your RAG: create 20 Q&A pairs from documents. Measure retrieval accuracy. Experiment with k=3 vs k=10 retrieved chunks.
Thu	**Advanced Retrieval**	Study hybrid search: BM25 (keyword) + semantic. Implement with LangChain EnsembleRetriever. Add cross-encoder reranking (BAAI/bge-reranker-base).	Build: Multi-query retrieval. Generate 3 query variations per user question. Deduplicate and re-rank. Measure improvement over naive RAG.
Fri	**RAG Evaluation**	Install RAGAS. Build evaluation dataset. Run faithfulness, answer relevancy, context precision metrics on your RAG system.	Standup: demo RAG system with live queries. Present RAGAS scores. Architecture review: draw your full RAG pipeline. Start production project.

Recommended Resources

These resources are drawn from official documentation and original research. Complete the highlighted readings before their corresponding daily session.

Type	Resource	Why It Matters
Paper	Retrieval-Augmented Generation for Knowledge-Intensive NLP (Lewis et al., 2020)	Original RAG paper
Paper	Advanced RAG: Survey of Techniques (2024)	Modern RAG taxonomy
Docs	LangChain RAG Tutorial (v0.3)	Official RAG implementation guide
Docs	ChromaDB Documentation	Primary vector DB for the course
Docs	RAGAS Documentation	RAG evaluation framework
Docs	Sentence-Transformers Docs (HuggingFace)	Local embeddings reference
Video	LlamaIndex RAG From Scratch Series (YouTube)	Deep dive into RAG patterns
Blog	Pinecone — What Is a Vector Database? (2024)	Vector DB concepts explained
Blog	LangChain Blog — Advanced RAG Techniques	Production RAG patterns
Repo	RAGAS GitHub Repository	Evaluation framework with examples
Blog	Jerry Liu (LlamaIndex) — 12 RAG Pain Points and Solutions	Common RAG problems and fixes

Hands-on Labs

Complete all three labs over the course of the week. Each lab builds toward the skills required for your Friday project submission.

Lab 3.1 — Semantic Search CLI

Embed 100 Wikipedia sentences using sentence-transformers. Build a CLI tool that takes a query, embeds it, and returns the most semantically similar sentences. Compare results between all-MiniLM-L6-v2 and bge-small-en.

Lab 3.2 — Naive RAG Pipeline

Build a complete RAG pipeline over 50 pages of PDF content: load, split, embed, store in ChromaDB with metadata, retrieve, and generate. Tune chunk size across 256, 512, and 1024 tokens and document the impact on answer quality.

Lab 3.3 — Hybrid Retrieval & RAGAS Evaluation

Implement hybrid search combining BM25 keyword search with semantic retrieval using LangChain's EnsembleRetriever, then add cross-encoder re-ranking. Evaluate the resulting pipeline with RAGAS, reporting faithfulness, answer relevancy, and context precision scores.

Weekly Assessment

Complete this self-assessment before Friday's standup. These questions test conceptual clarity, not memorisation.

Question Type	Question
Conceptual	What does an embedding represent? Why does cosine similarity work as a measure of semantic closeness?
Conceptual	Explain the tradeoffs between chunk size, retrieval accuracy, and context window usage in RAG.
Conceptual	What is the difference between naive RAG and hybrid search? Why might hybrid outperform pure semantic search?
Technical	Name three RAGAS metrics and explain what each one measures about a RAG system.
Technical	What does a cross-encoder re-ranker do, and why is it applied after initial retrieval rather than instead of it?
Design	Design a RAG architecture for a multi-tenant SaaS product where each customer's documents must remain isolated.

Weekly Standup Requirements

Every Friday standup follows the same structure. Prepare each item in advance.

Item	Detail
Demo	Live demo of your RAG system (2 minutes). Run at least 3 real queries against your indexed documents.
Technical Question	Explain one retrieval or evaluation concept from the week that changed how you think about RAG.
Architecture Review	Draw your full RAG pipeline: ingestion path and query path, end to end.
Code Review	Share your RAGAS evaluation results. What scored well? What scored poorly, and why?
Blockers	Name one retrieval quality or chunking challenge you faced. How did you solve it (or plan to)?

Week 3 Projects

Choose exactly one Intermediate project and one Production project to complete by Friday. Both are submitted via GitHub and demonstrated in the Friday standup.

Category 1 Projects — Choose One

Project 3-I-A: Personal Knowledge Base

Personal Knowledge Base	
Difficulty	Intermediate
Description	Build a personal Q&A system from your own documents (notes, articles, PDFs). Supports multi-document ingestion, filtering by source, and streaming responses.
Architecture	Document Loader (multi-format) → Text Splitter → Sentence-Transformers Embedder → ChromaDB → LangChain RetrievalQA → CLI
Skills Learned	Multi-format document loading, embedding pipelines, ChromaDB, retrieval chains
Deliverables	CLI tool ingesting 20+ documents, 15 Q&A examples, README with architecture
Evaluation Criteria	Retrieves relevant context. Answers grounded in documents. Source citations shown.

Project 3-I-B: Product Manual Assistant

Product Manual Assistant	
Difficulty	Intermediate
Description	Build a RAG chatbot over a product manual (choose any open-source manual: Linux man pages, Python docs, Django docs). Answers technical questions with page references.
Architecture	PDF/HTML Loader → Recursive Splitter → Local Embeddings → FAISS → Retrieval Chain → Streamlit UI
Skills Learned	Document chunking strategies, FAISS, Streamlit, source attribution
Deliverables	Streamlit app, 20 test questions with answers and source references, accuracy report
Evaluation Criteria	80%+ answer accuracy. Sources always cited. UI is clean and usable.

Project 3-I-C: Hybrid Search Engine

Hybrid Search Engine	
Difficulty	Intermediate

Description	Build a hybrid search system combining BM25 (keyword) and semantic search over a news corpus. Compare search quality with and without re-ranking.
Architecture	News Dataset → BM25 Index (rank_bm25) + Vector Index (ChromaDB) → EnsembleRetriever → Cross-Encoder Re-ranker → Results Display
Skills Learned	BM25 implementation, ensemble retrieval, cross-encoder reranking, search evaluation
Deliverables	CLI search tool, comparison study (BM25 vs semantic vs hybrid), evaluation on 30 queries
Evaluation Criteria	Hybrid outperforms either method alone. Re-ranking improves top-1 accuracy. Study is rigorous.

Category 2 Projects — Choose One

Project 3-P-A: Enterprise Document Intelligence Platform

Enterprise Document Intelligence Platform	
Difficulty	Production
Description	Build a multi-tenant document intelligence API. Users upload documents, which are processed and indexed. Query endpoint returns answers with confidence scores, source chunks, and page numbers. Includes admin dashboard.
Architecture	FastAPI + Background Worker (asyncio) → Document Processing Pipeline → ChromaDB (namespaced per tenant) → Advanced RAG (hybrid + rerank) → REST API → Streamlit Admin Dashboard → Docker Compose
Skills Learned	Multi-tenancy, async processing, REST API design, background tasks, ChromaDB namespacing, admin UI
Deliverables	Docker-deployed platform, API docs, demo with 3 tenant datasets, RAGAS evaluation report, GitHub repo
Evaluation Criteria	Multi-tenancy works correctly. RAGAS scores > 0.7 on test set. Docker compose works. API is fully documented.

Project 3-P-B: Real-Time Research Engine

Real-Time Research Engine	
Difficulty	Production
Description	Build a research engine that combines RAG over a local knowledge base with live web search results. Dynamically decides whether to search web or use local knowledge. Produces cited reports.
Architecture	Query → Knowledge Router (local vs web) → Dual Retrieval (ChromaDB + Web Search Tool) → Context Merger → Re-ranker → Report Generator → FastAPI → Streamlit
Skills Learned	Knowledge routing, web search integration, context merging, dynamic retrieval strategy, citation management
Deliverables	Working research engine, 10 research reports on different topics, routing decision logs, deployed app
Evaluation Criteria	Routing decisions are correct 90%+ of time. Reports have proper citations. Hybrid approach outperforms single-source.

Project 3-P-C: RAG Evaluation Benchmark

RAG Evaluation Benchmark	
Difficulty	Production

Description	Build a comprehensive RAG evaluation framework that tests multiple RAG configurations (chunk sizes, embedding models, retrieval strategies) and produces statistical comparison reports. Useful as a standalone tool for future projects.
Architecture	Config YAML → Pipeline Builder → Parallel Evaluation Runner → RAGAS + Custom Metrics → Statistical Analyzer → HTML Report Generator → GitHub Pages
Skills Learned	Evaluation framework design, statistical analysis, parallel execution, HTML report generation, CI/CD
Deliverables	Published benchmark comparing 6+ RAG configurations, GitHub Actions CI, documentation, blog post about findings
Evaluation Criteria	Framework is reusable for new configurations. Statistical analysis is sound. Report is publishable quality.

Week 3 Completion Checklist

Confirm every item below before Friday's standup.

	Checklist Item
☐	All 5 daily learning sessions completed (Mon–Fri)
☐	Lab 3.1 — Semantic search CLI built and tested across two embedding models
☐	Lab 3.2 — Naive RAG pipeline built with chunk size experiments documented
☐	Lab 3.3 — Hybrid retrieval implemented and evaluated using RAGAS metrics
☐	Weekly Assessment questions answered in writing
☐	One Intermediate project chosen, built, and pushed to GitHub
☐	One Production project chosen, built, and pushed to GitHub
☐	Both projects have complete README files with setup instructions
☐	Full RAG pipeline architecture diagram prepared for Friday's standup review
☐	Demo rehearsed and ready to present in under 2 minutes